Введение

В рамках курсового проекта разработано игровое приложение в жанре «лабиринт с туманом войны» – «Эхо-лабиринт».

Цель курсового проекта – разработка программного

продукта, демонстрирующего применение процедурной генерации, алгоритмов поиска пути и звукового взаимодействия при создании интерактивного игрового приложения.

Приложение представляет собой однопользовательскую игру, в которой игрок перемещается по процедурно генерируемому лабиринту, используя звуковые волны для раскрытия окружения и поиска выхода, избегая при этом призраков.

Далее приведём краткое описание разделов пояснительной записки.

Раздел «Анализ задачи» содержит постановку задачи, описание предметной области и аналогов, входные и выходные данные, инструменты разработки, а также требования к аппаратным характеристикам.

Раздел «Проектирование задачи» описывает организацию данных, пользовательский интерфейс и алгоритмы обработки информации.

Раздел «Реализация» описывает программные модули, их структуру и аннотацию файлов проекта.

Раздел «Тестирование» содержит функциональное тестирование всех операций приложения от запуска до выхода.

Раздел «Руководство пользователя» описывает назначение программы, порядок запуска, а также инструкции по работе с интерфейсом.

«Заключение» содержит итоги работы и краткое описание применённых методов.

«Список использованных источников» содержит перечень литературы и ресурсов, использованных при разработке.

В приложении к пояснительной записке будет приведён листинг программы.

1 Анализ задачи

1.1 Постановка задачи

Наименование задачи: «Разработка игрового приложения «Эхо-лабиринт».

Цель разработки: создание игрового приложения в жанре «туманный лабиринт», демонстрирующего практическое применение процедурной генерации лабиринтов, алгоритма тумана войны и звуковой механики взаимодействия с окружением.

Назначение: игровое приложение предназначено для развлечения, а также развития пространственного мышления и реакции в условиях ограниченной видимости. Помимо основного геймплея, проект демонстрирует применение графической библиотеки SFML, процедурной генерации с использованием алгоритма обхода в глубину и систему тумана войны с затуханием.

Предметная область: жанр «лабиринт» – поджанр приключенческих игр, в котором игрок исследует процедурно генерируемые уровни, ориентируясь в условиях ограниченной видимости. Ключевая механика приложения – «звуковое эхо»: игрок кликает мышью, создавая звуковую волну, которая распространяется по лабиринту и кратковременно раскрывает его структуру.

Периодичность использования: зависит от потребности пользователя, может применяться ежедневно в режиме одиночной игры.

Источники и способы получения данных: игровой процесс формируется на основе процедурно генерируемого лабиринта. Пользователь воздействует на процесс посредством мыши и клавиатуры.

Выходная информация: графическое отображение игрового процесса в окне: процедурно генерируемый лабиринт с туманом войны, анимированные призраки трёх типов, звуковые волны, информационная панель (уровень, очки, таймер, полоска перезарядки волны).

Входная информация: нажатия кнопок мыши (левая кнопка – запуск звуковой волны, нажатие кнопок интерфейса) и нажатия клавиш клавиатуры (клавиши-стрелки или WASD – движение, R – перезапуск уровня, Esc – главное меню, F11 – полноэкранный режим).

1.2 Инструменты разработки

Для разработки данного курсового проекта выбрана среда разработки Microsoft Visual Studio 2022, так как она обеспечивает удобную интеграцию с системой сборки CMake, предоставляет инструменты отладки и совместима с компилятором MSVC.

Язык программирования – C++17, обеспечивающий поддержку современных возможностей языка: структурных привязок, std::unique_ptr, лямбда-выражений, а также совместимых с SFML 3 типов данных.

Графическая библиотека – SFML 3 (Simple and Fast Multimedia Library) – кросс-платформенная библиотека для работы с графикой, вводом-выводом и звуком на C++. Выбор обусловлен простотой API, наличием встроенных модулей для работы с примитивами, звуком и текстом.

Система сборки – CMake – кросс-платформенная система автоматической сборки, позволяющая управлять зависимостями проекта и конфигурацией компиляции.

1.3 Требования к приложению

На этапе исследования предметной области был установлен ряд требований, предъявляемых к разрабатываемой задаче.

При проектировании интерфейса необходимо учесть следующие требования:

- интуитивно понятное управление посредством мыши и клавиатуры;
- наглядное отображение уровня, очков и таймера на информационной панели;
- полоска перезарядки звуковой волны;
- визуальное различие трёх типов призраков;
- наличие главного меню с кнопками «Играть», «Настройки», «Справка», «Об авторе», «Выход»;
- наличие экранов окончания игры.

Функциональные требования к приложению:

- процедурная генерация лабиринта на каждом уровне алгоритмом обхода в глубину;
- механика звуковой волны: волна распространяется от точки клика, раскрывая структуру лабиринта на 2 секунды;
- туман войны: посещённые клетки подсвечиваются в течение 5 секунд, затем гаснут;
- три типа призраков с различным поведением (обычный, быстрый, охотник);
- нарастание сложности: с каждым уровнем добавляется призрак;
- настраиваемые параметры: скорость призраков, перезарядка волны, начальное количество призраков, схема управления;
- звуковое сопровождение игрового процесса.

2 Проектирование задачи

2.1 Организация данных

Проектирование задачи является важным и ответственным этапом в разработке любого приложения, поскольку на этом этапе определяется структура данных и логика взаимодействия компонентов.

В разрабатываемой программе используются три вида данных.

Первым видом являются данные, задаваемые разработчиком на этапе реализации. К ним относятся: константы размеров лабиринта (ROWS, COLS, TILE), параметры волны (WAVE_SPEED, WAVE_MAX_R, REVEAL_TIME), скорость игрока и призраков, радиусы столкновений. Эти данные хранятся в виде констант типа constexpr.

Вторым видом данных является динамическое состояние игры: позиция игрока, список активных призраков и волн, матрица лабиринта maze[ROWS][COLS], матрица таймеров тумана войны visitedTimer[ROWS][COLS], счёт, текущий уровень и таймер уровня. Эти данные хранятся в локальных переменных и структурах внутри функции main.

Третьим видом данных являются вычисляемые данные: позиции объектов после обновления, результаты проверки столкновений, видимость клеток по таймеру. Эти данные вычисляются на каждом кадре игрового цикла и не хранятся постоянно.

Основными структурами хранения объектов являются статические двумерные массивы для лабиринта и тумана войны, а также динамические одномерные массивы std::vector для призраков, волн и частиц. Выбор обусловлен необходимостью добавления и удаления объектов в течение игровой сессии.

2.2 Процессы

Согласно требованиям, рассмотренным в разделе «Анализ задачи», была определена следующая структура игрового процесса.

Главный игровой цикл выполняется непрерывно и состоит из трёх этапов: обработка событий, обновление состояния, отрисовка кадра.

На этапе обновления состояния выполняются следующие действия:

- обновление позиции игрока на основе нажатых клавиш с проверкой столкновений со стенами лабиринта;
- обновление таймера тумана войны: клетки в окрестности игрока получают 5 секунд видимости, все таймеры убывают на dt;
- обновление волн: радиус каждой волны увеличивается, клетки в зоне видимости получают таймер раскрытия REVEAL_TIME;

– обновление призраков: каждый призрак выбирает направление согласно своему типу поведения и перемещается с проверкой стен;
– проверка условий: столкновение с призраком завершает игру, достижение выхода запускает переход на следующий уровень.

Генерация лабиринта выполняется алгоритмом обхода в глубину (recursive backtracker). Алгоритм начинается с клетки (1,1), рекурсивно обходит соседние клетки через шаг 2, прорубая проходы, до исчерпания всех клеток. Дополнительно случайным образом удаляются до 30 внутренних стен для создания альтернативных маршрутов.

Механика звуковой волны. Волна распространяется от точки клика со скоростью WAVE_SPEED пикселей в секунду. Для каждой клетки лабиринта, находящейся в радиусе волны, выполняется проверка прямой видимости (hasLineOfSight): если волна может достичь клетки без пересечения стен, клетке назначается таймер раскрытия на REVEAL_TIME секунд.

Поведение призраков определяется их типом. Обычный призрак с частотой 60% движется к игроку, в остальных случаях – случайно. Быстрый призрак движется хаотично с двойной скоростью. Охотник постоянно обновляет направление в сторону игрока. Все типы призраков убегают от активных волн.

2.3 Описание внешнего пользовательского интерфейса

Пользовательский интерфейс приложения выполнен в строгом чёрно-белом стиле, соответствующем тематике звукового эха в темноте.

Главное меню содержит название игры – «Эхо-лабиринт», анимированный фон из восходящих частиц, а также следующие кнопки:

– «Играть» – запуск игровой сессии с первого уровня;
– «Настройки» – экран управления параметрами игры;
– «Справка» – экран с описанием управления и механик;
– «Об авторе» – информация о разработчике;
– «Выход» – завершение работы приложения.



Рисунок 1 – Главное меню

Игровой экран занимает разрешение WIN_W × WIN_H пикселей. Лабиринт отображается на чёрном фоне: стены и пол видны только в радиусе звуковой волны или в зоне тумана войны. В верхней части экрана расположена информационная панель, содержащая:

– текущий уровень и количество очков (левый верхний угол);
– обратный таймер и количество призраков (правый верхний угол);
– полоску перезарядки волны по центру нижней части экрана.

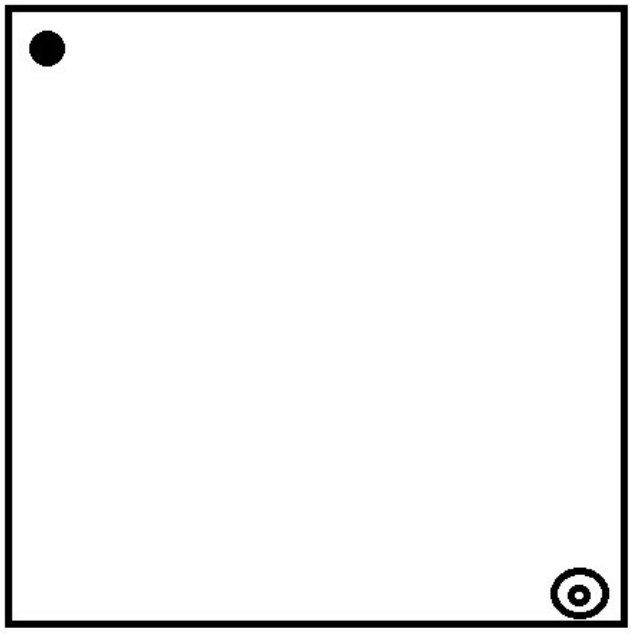


Рисунок 2 – Игровой экран

Экран настроек позволяет изменить следующие параметры:

- скорость призраков: Медленно / Обычно / Быстро;
- перезарядка волны: 1 / 2 / 3 секунды;
- начальное количество призраков: 1 / 2 / 3;
- схема управления: стрелки / WASD;
- полноэкранный режим.

3 Реализация

3.1 Реализация проекта

3.1.1 Структура программы

Данный курсовой проект содержит 1 файл реализации – main.cpp. Далее рассмотрим назначение каждого логического блока программы.

1 Блок глобальных констант – определяет все параметры игры: размеры окна (WIN_W, WIN_H), размер тайла (TILE), скорости игрока и призраков, радиусы столкновений, время перезарядки волны и время тумана войны. Использование constexpr гарантирует вычисление значений на этапе компиляции.

2 Пространство имён Palette – содержит все цвета приложения в виде именованных констант типа sf::Color. Реализована полностью чёрно-белая палитра: оттенки серого с различной прозрачностью, обеспечивающая атмосферный стиль звукового эха.

3 Функция generateMaze – реализует процедурную генерацию лабиринта алгоритмом обхода в глубину с использованием стека std::stack<Cell>. После генерации случайным образом удаляются до EXTRA_WALLS_REMOVED дополнительных стен для создания альтернативных маршрутов.

4 Функция hasLineOfSight – проверяет прямую видимость между точкой происхождения волны и центром клетки лабиринта путём трассировки луча с шагом 0.4 тайла.

5 Структура Sounds – обеспечивает загрузку и воспроизведение звуковых эффектов через SFML Audio. Буферы хранятся в sf::SoundBuffer, звуки создаются через std::unique_ptr<sf::Sound> только после успешной загрузки буфера.

6 Структура Ghost – описывает призрака: позицию, направление движения, тип поведения (Normal, Fast, Hunter), фазу покачивания для анимации. Метод update реализует логику выбора направления, избегания стен и притяжения к волнам.

7 Функция drawGhost – реализует ручную отрисовку призрака через sf::VertexArray. Форма тела зависит от типа: ширина и высота, количество зубцов нижней части (2–4), форма глаз. Охотник отмечен ромбом над головой, быстрый – следом движения.

8 Структура UI – вспомогательный класс для отрисовки текста и кнопок. Метод text отрисовывает sf::Text с заданным шрифтом и цветом, метод button – прямоугольную кнопку с подсветкой при наведении.

9 Главная функция main – содержит игровой цикл, обработку событий, логику состояний и рендеринг. Разделена на блоки: главное меню, логика игры, рендер. Состояния игры управляются перечислением GameState.

3.1.2 Структура и описание разработанных процедур и функций

Описание разработанных функций и структур находится в таблице 1.

Таблица 1 – Процедуры и функции

Имя процедуры (функции)	**В каком блоке находится**	**За каким компонентом закреплена**	**Назначение**
1 generateMaze()	main.cpp	–	Процедурная генерация лабиринта алгоритмом обхода в глубину
2 hasLineOfSight()	main.cpp	–	Проверка прямой видимости между точкой волны и клеткой лабиринта
3 drawGhost()	main.cpp	–	Ручная отрисовка призрака через sf::VertexArray по типу
4 createWindow()	main.cpp	–	Создание sf::RenderWindow в оконном или полноэкранном режиме
5 Ghost::update()	struct Ghost	–	Обновление позиции призрака: выбор направления, избегание стен, притяжение к волне
6 Ghost::collidesWall()	struct Ghost	–	Проверка столкновения призрака со стенами лабиринта
7 Ghost::avoidWalls()	struct Ghost	–	Поиск направления обхода стены путём перебора углов поворота
8 Ghost::speed()	struct Ghost	–	Возврат скорости призрака в зависимости от типа и множителя
9 Sounds::load()	struct Sounds	–	Загрузка звуковых файлов из папки assets/sounds/
10 Sounds::playWave()	struct Sounds	–	Воспроизведение звука звуковой волны

Имя процедуры (функции)	В каком блоке находится	За каким компонентом закреплена	Назначение
11 Sounds::playDie()	struct Sounds	–	Воспроизведение звука поражения
12 Sounds::playExit()	struct Sounds	–	Воспроизведение звука перехода на следующий уровень
13 UI::text()	struct UI	–	Отрисовка текстовой строки с заданным шрифтом, размером и цветом
14 UI::button()	struct UI	–	Отрисовка кнопки интерфейса с подсветкой при наведении курсора
15 resetLevel()	main()	–	Сброс уровня: генерация лабиринта, очистка волн и частиц, спавн призраков
16 startGame()	main()	–	Запуск новой игры: сброс счёта и уровня, вызов resetLevel
17 respawnGhosts()	main()	–	Создание призраков с распределением типов по уровню
18 spawnParticles()	main()	–	Создание частиц для визуальных эффектов
19 applyFullscreen()	main()	–	Пересоздание окна при переключении полноэкранного режима
20 collidesPlayer()	main()	–	Проверка столкновения игрока со стенами лабиринта

3.1.3 Описание использованных компонентов

Описание использованных для разработки приложения компонентов приводится в таблице 2.

Таблица 2 – Использованные компоненты

Компонент	**На каком модуле расположен**	**Назначение**
1 sf::RenderWindow	main()	Главное окно приложения; управляет циклом событий и отрисовкой
2 sf::CircleShape	drawGhost(), main()	Отрисовка игрока, ауры, волн, глаз призраков, частиц
3 sf::RectangleShape	main(), UI::button()	Отрисовка тайлов лабиринта, кнопок, полосок HUD, оверлеев
4 sf::VertexArray	drawGhost()	Ручная отрисовка тела призрака через треугольный веер
5 sf::ConvexShape	drawGhost()	Отрисовка ромба-маркера над призраком-охотником
6 sf::Text	UI::text(), main()	Отображение текста HUD, меню, подсказок, экранов завершения
7 sf::Font	struct UI	Загрузка шрифта Arial для отрисовки всех текстовых элементов
8 sf::View	main()	Масштабирование игрового вида с сохранением пропорций (letterbox)
9 sf::Clock	main()	Измерение времени кадра (deltaTime) для независимого от FPS движения
10 sf::SoundBuffer	struct Sounds	Хранение загруженных звуковых данных в памяти
11 sf::Sound	struct Sounds	Воспроизведение звуковых эффектов из sf::SoundBuffer
12 sf::VideoMode	createWindow()	Получение разрешения рабочего стола для полноэкранного режима

Компонент	На каком модуле расположен	Назначение
13 std::stack<Cell>	generateMaze()	Стек для итеративной реализации алгоритма обхода в глубину
14 std::vector<Ghost>	main()	Динамический список призраков с возможностью добавления в процессе игры
15 std::vector<Wave>	main()	Динамический список активных звуковых волн
16 std::vector<Particle>	main()	Динамический список частиц для визуальных эффектов
17 std::unique_ptr<sf::Sound>	struct Sounds	Управление временем жизни объекта sf::Sound после загрузки буфера

3.2 Спецификация программы

Точное название проекта и его состав приводится в таблице 3.

Таблица 3 – Спецификация программы

Имя файла	Назначение
1 EchoMaze.exe	Исполняемый файл проекта
2 main.cpp	Единственный файл реализации: вся логика игры, рендер, меню
3 CMakeLists.txt	Файл системы сборки CMake; описывает зависимости и параметры компиляции
4 assets/fonts/arial.ttf	Шрифт Arial для отрисовки текстовых элементов интерфейса
5 assets/sounds/wave.ogg	Звуковой эффект запуска звуковой волны
6 assets/sounds/die.ogg	Звуковой эффект поражения (касание призрака)
7 assets/sounds/exit.ogg	Звуковой эффект успешного перехода на следующий уровень

4 Тестирование

При разработке данного программного продукта возникающие ошибки и недоработки исправлялись в процессе реализации. По завершении разработки было проведено функциональное тестирование, которое должно гарантировать корректную работу всех элементов программы.

Отчёт о результатах тестирования представлен в таблице 2.

Таблица 2 – Отчёт о результатах тестирования

Тест	**Ожидаемый результат**	**Фактический результат**	**Итог**
Запуск приложения	Откроется главное меню	Открывается главное меню	Выполнено
Кнопка «Играть»	Начнётся игра с первого уровня	Игра начинается с первого уровня	Выполнено
Кнопка «Настройки»	Откроется экран настроек	Открывается экран настроек	Выполнено
Кнопка «Справка»	Откроется экран справки	Открывается экран справки	Выполнено
Кнопка «Об авторе»	Откроется экран об авторе	Открывается экран об авторе	Выполнено
Кнопка «Выход»	Приложение закроется	Приложение закрывается	Выполнено
Движение стрелками	Игрок перемещается по лабиринту	Игрок перемещается по лабиринту	Выполнено
Движение WASD	Игрок перемещается (при выборе WASD в настройках)	Игрок перемещается корректно	Выполнено
ЛКМ – звуковая волна	Волна распространяется, раскрывая лабиринт	Волна распространяется, раскрывает лабиринт	Выполнено
Волна через стену	Волна не проходит сквозь стену	Волна не проходит сквозь стену	Выполнено
Перезарядка волны	Повторный выстрел недоступен до окончания	Повторный выстрел недоступен до окончания	Выполнено

Тест	Ожидаемый результат	Фактический результат	Итог
	перезарядки	перезарядки	
Столкновение с призраком	Игра завершается, отображается экран поражения	Игра завершается, отображается экран поражения	Выполнено
Достижение выхода	Игрок переходит на следующий уровень	Происходит переход на следующий уровень	Выполнено
Переход уровня – fade	Плавное затемнение с надписью «Уровень N»	Отображается плавное затемнение с надписью	Выполнено
Туман войны – посещённые клетки	Клетки подсвечены 5 секунд, затем гаснут	Клетки подсвечены 5 секунд, затем гаснут	Выполнено
Поведение обычного призрака	Призрак периодически движется к игроку	Призрак периодически движется к игроку	Выполнено
Поведение быстрого призрака	Призрак движется хаотично с высокой скоростью	Призрак движется хаотично с высокой скоростью	Выполнено
Поведение охотника	Призрак упорно преследует игрока	Призрак упорно преследует игрока	Выполнено
Убегание призрака от волны	Призрак меняет направление при попадании волны	Призрак меняет направление при попадании волны	Выполнено
Нарастание сложности	С каждым уровнем добавляется призрак	С каждым уровнем добавляется призрак	Выполнено
Появление нового призрака по таймеру	Каждые 20 с появляется новый призрак	Каждые 20 с появляется новый призрак	Выполнено
Клавиша R – перезапуск уровня	Текущий уровень начинается заново	Текущий уровень начинается заново	Выполнено
Клавиша Esc – главное меню	Происходит возврат в главное меню	Происходит возврат в главное меню	Выполнено
F11 –	Окно переходит в	Окно переходит в	Выполнено

Тест	Ожидаемый результат	Фактический результат	Итог
полноэкранный режим	полноэкранный режим	полноэкранный режим	
Смена управления на WASD	В настройках переключается управление	Управление переключается корректно	Выполнено
Изменение скорости призраков	Скорость изменяется согласно настройке	Скорость изменяется согласно настройке	Выполнено
Изменение перезарядки волны	Перезарядка изменяется (1/2/3 сек)	Перезарядка изменяется корректно	Выполнено
Звук волны	При клике воспроизводится звуковой эффект	Звуковой эффект воспроизводится	Выполнено
Звук смерти	При поражении воспроизводится звук	Звук воспроизводится при поражении	Выполнено
Звук перехода уровня	При достижении выхода воспроизводится звук	Звук воспроизводится при достижении выхода	Выполнено

По результатам тестирования все функциональные требования выполнены в полном объёме. Ошибок, приводящих к некорректной работе программы, не обнаружено. Логика игрового процесса, туман войны, механика волны, поведение призраков, воспроизведение звука и смена состояний работают корректно.

5 Руководство пользователя

5.1 Общие сведения о программном продукте

Цель данного программного продукта — предоставить пользователю интерактивный игровой опыт в жанре «лабиринт с туманом войны». Игра называется «Эхо-лабиринт» и выполнена в строгом чёрно-белом стиле, создающем атмосферу темноты и звукового ориентирования.

Назначение программы — развлечение пользователя, развитие пространственного мышления и реакции через игровую механику ориентирования по звуку.

Область применения: индивидуальное использование на персональном компьютере. Приложение рассчитано на широкий круг пользователей, интересующихся головоломками и играми в жанре «рогалик».

Основные возможности программы:

- процедурно генерируемые лабиринты на каждом уровне;
- механика звуковой волны для исследования окружения;
- туман войны с плавным затуханием;
- три типа призраков с различным поведением;
- нарастание сложности с каждым уровнем;
- настраиваемые параметры игры;
- звуковое сопровождение.

5.2 Инсталляция

Для установки программы «Эхо-лабиринт» необходимо запустить файл EchoMaze_setup.exe. После запуска появится окно установки приложения «Эхо-лабиринт». Затем достаточно следовать приведённой инструкции установщика.

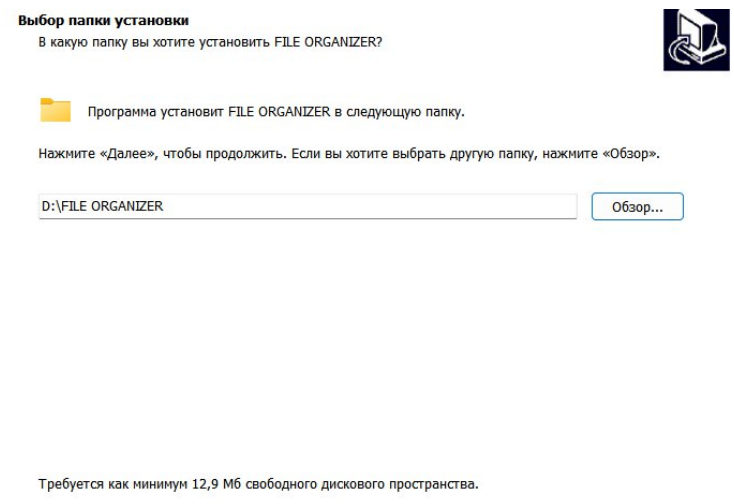


Рисунок 3 – Окно установщика

5.3 Выполнение программы

5.3.1 Запуск программы

Для запуска программы необходимо дважды щёлкнуть левой кнопкой мыши по исполняемому файлу приложения. После запуска на экране отображается главное меню игры «Эхо-лабиринт».

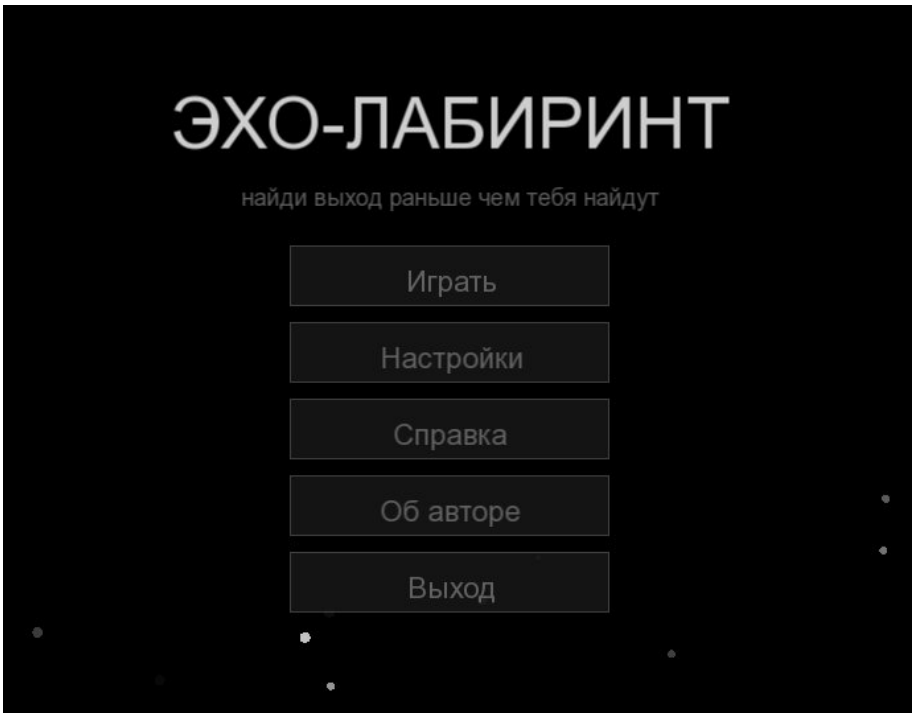


Рисунок 4– Главное меню программы

5.3.2 Инструкции по работе с программой

Главное меню. После запуска приложения отображается главное меню с анимированным фоном из восходящих частиц и кнопками управления. Для начала игры необходимо нажать кнопку «Играть». Кнопка «Настройки» открывает экран с параметрами игры. Кнопка «Справка» открывает экран с описанием управления и типов призраков. Кнопка «Об авторе» отображает информацию о разработчике. Кнопка «Выход» завершает работу приложения.

Игровой экран. После нажатия «Играть» открывается игровой экран. Лабиринт отображается на чёрном фоне — изначально полностью скрыт. Для исследования лабиринта необходимо кликнуть левой кнопкой мыши в любой точке экрана: от этой точки распространится звуковая волна, кратковременно раскрывая структуру лабиринта.

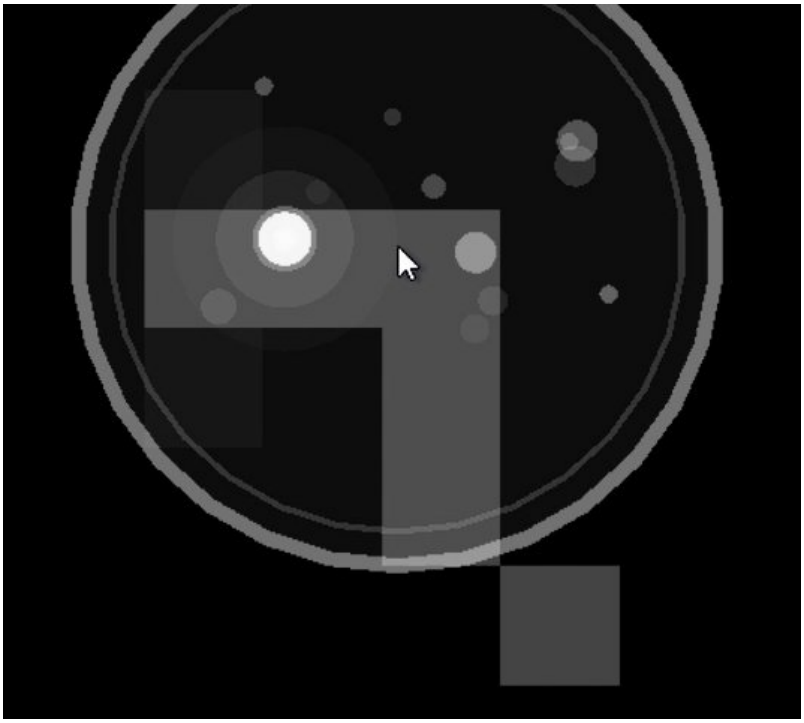


Рисунок 5 – Игровой экран с активной звуковой волной

Управление. Для перемещения игрока используются клавиши-стрелки (или WASD при соответствующей настройке). Игрок обозначен белым кружком с пульсирующей аурой. Выход с уровня обозначен мерцающим белым кружком в правом нижнем углу лабиринта.

Туман войны. Клетки лабиринта, через которые прошёл игрок, подсвечиваются слабым серым цветом в течение 5 секунд, после чего плавно гаснут. Это позволяет ориентироваться в ранее посещённых областях, не раскрывая лабиринт полностью.

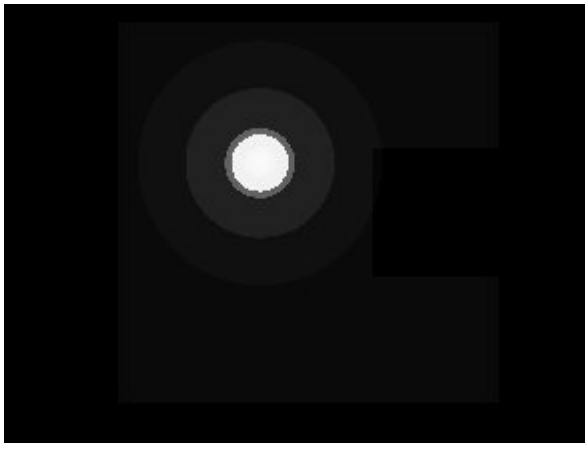


Рисунок 6 – Туман войны: посещённые клетки

Призраки. В лабиринте присутствуют призраки трёх типов, различающиеся внешним видом и поведением. Серый призрак с тремя зубцами — обычный — периодически движется в сторону игрока. Белый призрак с четырьмя зубцами — быстрый — хаотично перемещается с высокой скоростью. Тёмный призрак с двумя зубцами и ромбом над головой — охотник — упорно преследует игрока. Все призраки убегают при попадании звуковой волны.

Смена уровней. При достижении выхода экран плавно затемняется, отображается надпись «Уровень N», после чего генерируется новый лабиринт с увеличенным числом призраков.

Пауза и выход. Клавиша Esc возвращает в главное меню. Клавиша R перезапускает текущий уровень без сброса счёта. Клавиша F11 переключает полноэкранный режим.

5.3.3 Завершение работы с программой

Для завершения работы с программой необходимо нажать кнопку «Выход» в главном меню или закрыть окно приложения стандартной кнопкой закрытия окна.

5.4 Использование системы справочной информации

В состав программного продукта входит файл справки EchoMaze.chm, разработанный в среде HelpNDoc. Файл справки расположен в папке assets/help/ в директории игры.

Для открытия справки необходимо нажать кнопку «Справка» в главном меню игры. Приложение автоматически откроет файл EchoMaze.chm с помощью системного средства просмотра справки Windows.

Файл справки содержит следующие разделы:

О программе — общие сведения об игре, версия и используемые технологии;

Об авторе — информация о разработчике.

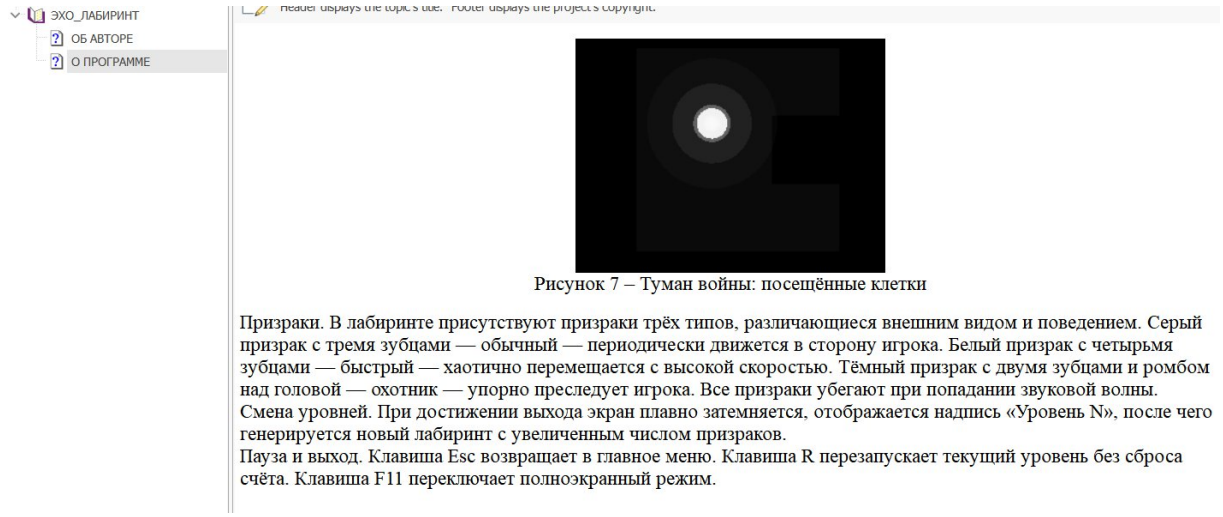


Рисунок 7 – Файл справки EchoMaze.chm

Заключение

В результате выполнения курсового проекта разработан программный продукт – игровое приложение «Эхо-лабиринт» в жанре «лабиринт с туманом войны».

В ходе работы выполнено следующее:

- изучена предметная область жанра «лабиринт» и рассмотрены аналоги;
- реализована процедурная генерация лабиринта алгоритмом обхода в глубину с дополнительным удалением стен;
- разработана механика звуковой волны с трассировкой прямой видимости (hasLineOfSight);
- реализован туман войны с плавным затуханием на основе массива таймеров visitedTimer;
- разработаны три типа призраков с уникальным внешним видом (ручная отрисовка через sf::VertexArray) и поведением;
- реализовано нарастание сложности: с каждым уровнем добавляется призрак, увеличивается скорость;
- разработан полноценный пользовательский интерфейс в чёрно-белом стиле: главное меню, экран настроек, справка, экран поражения;
- интегрировано звуковое сопровождение: три звуковых эффекта (волна, поражение, переход уровня);
- реализованы атмосферные эффекты: пульсация игрока, след движения, плавный переход уровней;
- проведено функциональное тестирование – 30 тестов, все выполнены успешно.

Поставленная задача выполнена в полном объёме. Приложение функционирует корректно, удовлетворяет всем сформулированным требованиям и обеспечивает законченный игровой процесс. Возможными направлениями дальнейшего развития проекта являются: реализация алгоритма поиска пути A* для охотника, добавление предметов в лабиринте, мини-карты посещённых клеток и таблицы рекордов с сохранением в файл.

Список использованных источников

1. Официальная документация SFML 3.1.0 [Электронный ресурс]. – Режим доступа: https://www.sfml-dev.org/documentation/3.1.0/ – Дата доступа: 25.05.2026.
2. Справочник языка C++ [Электронный ресурс]. – Режим доступа: https://en.cppreference.com – Дата доступа: 08.06.2026.
3. Учебный курс по C++ [Электронный ресурс]. – Режим доступа: https://www.learncpp.com – Дата доступа: 01.06.2026.
4. Репозиторий библиотеки SFML [Электронный ресурс]. – Режим доступа: https://github.com/SFML/SFML – Дата доступа: 29.05.2026.
5. Buck, J. Mazes for Programmers: Code Your Own Twisty Little Passages. – The Pragmatic Bookshelf, 2015. – 286 с.
6. Gregory, J. Game Engine Architecture. 3rd ed. – CRC Press, 2018. – 1200 с.
7. Руководство по стандарту C++ Core Guidelines [Электронный ресурс]. – Режим доступа: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines – Дата доступа: 12.06.2026.